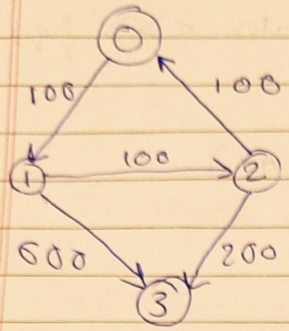
Name : Sudhanshu

UID : 23BCS12388 Expt - 7

Class : KRG - 1 - A

Q



Algorithm :-

1) Build Adjacency list from flights

2) Initialize ~~queue~~ min-heap of format
   [cost, node, stops]
   [0, src, 0]

3) Initialize distance array, dist[src] = 0
   dist[others] = ∞

4) Process nodes while queue is not empty :-

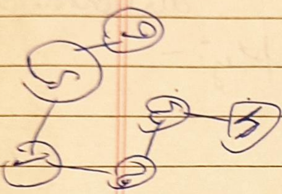
   (i) Pop top value
   (ii) if stops > k continue;
   (iii) for each neighbor :
        if cost + edgecost < dist [adjnode] :
            update dist [adjnode]
            push (newcost, adjnode, stops + 1)

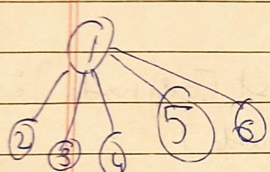
5) Return (dist[dest] == ∞ ? -1 : dist[dest])

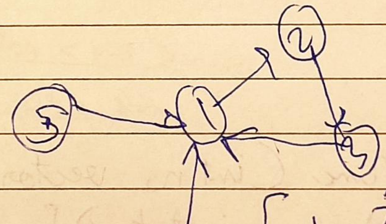
Code

```
int func (int n, vector<vector<int>> & flights, int src, int
    dest , int k ){
    vector <vector <pair<int, int>> > adj(n);
    for (auto & f: flights ){
        adj[f[0]] . push_back ({f[1], f[2]});
    }
```

```
priority_queue < tuple <int, int, int>, vector < tuple<
        int, int, int >> , greater <>> pq;
pq.push ({0, src, 0});

vector<int> dist (n, INT_MAX);
 dist [0] = 0;
while (!pq.empty()){

        auto [cost, node, stops] = pq.top();
        pq.pop();
        if ( stops > K  || stops > dist [node]) continue;
        if (node == dest) return cost;

        dist[ node] = stops;

        for (auto &x: adj [node]){
             int next = x.first;
             int val = x.second;

             pq.push ({cost + val, next, stops+1});
        }
   }
   return -1;
}
```







[1, 2, 3, 4]

1 1, 3 4

1, 1, 4

i, j, K

```
if (dist [i] [k] + dist [k] [j] < dist [i, j]){

        dist[i, j] = dist [i] [k] + dist [k] [j];
}
```

5 1 1 4